Assignment 6:
Order Statistics Analysis and Basic Data Structures

Shreeya Gadgil

For MSCS 532 – Algorithms and Data Structures

A) Order Statistics: Deterministic vs Randomized Selection:

Introduction:

The selection problem asks for the k-th smallest element of an unsorted array of n elements. Sorting solves it in O(nlogn), but selecting can be done in linear time. Here we'll compare the deterministic(Median of Medians) and randomized quickselect.

i. Implementation:

Both algorithms share the three-way partition and differ in how they choose the pivot.

- **Three-way partition:**
  A two-way partition can degrade to $\Theta n\hat{}2$ when many keys equal the pivot, because the equal keys all land on one side. Three-way splits a[low.....high] into less than pivot, equal to the pivot and greater than the pivot in one pass. If the target falls in the middle block, we go to return, so an array of all equal keys finishes in a single partition loop.
- **Randomized QuickSelect:**
  Pick a pivot uniformly at random, then partition and continue only in the side that holds rank k. The loop is iterative, so there is no recursion in the picture.
- **Deterministic selection:**
  The pivot is chosen by the BFPRT rule by first splitting the range into groups of 5 and insertion sort each group. Then move each group's median to the front of the range. Then recursively select the median of those medians as the pivot. Three-way partition around it and continue in the side holding rank k.

ii. Time Complexity:

- **Best Case:** For randomized Quickselect, the best case is the run in which the very first random pivot is the k-th smallest element itself, or more generally any pivot whose block of equal keys contains rank k. The three-way partition then places the target in the == block and the search ends after a single pass. An array in which every element is equal is the extreme example: any pivot works. Median of Medians reaches its best case in the same situation, when its carefully computed pivot happens to be the answer, for instance when k is exactly the rank of the median of medians or when all keys are equal.
  Even the luckiest run cannot beat Ω(n), for a simple reason: an algorithm that skips any element could be fooled, because that unseen element might be the answer. So every element must be compared at least once. For randomized Quickselect, the single partition does exactly that, about n comparisons. Median of Medians has to do more, because it pays for computing its pivot before it can partition. Even if that first pivot is the answer, the cost is one round of group sorting and partitioning plus the recursive median-of-medians call on n/5 elements:

$$T_{best}(n) = T_{best}\left(\frac{n}{5}\right) + cn = \frac{5}{4}cn$$

  Both algorithms are therefore Θ(n) in the best case; randomized Quickselect simply has the smaller constant.
- **Average Case:** For randomized QuickSelect, the average case means averaged over the algorithm's own coin flips, so the bound holds for every input, including sorted and adversarial ones. Call a pivot good if its rank lies in the middle half of the current range (between the 25th and 75th percentile). A uniform random pivot is good with probability

1/2, and a good pivot leaves at most 3n/4 elements. Group the run into phases, with phase j lasting while the range holds between (3/4)^(j+1)·n and (3/4)^j·n elements. Each partition in phase j costs at most c(3/4)^j·n, and a phase ends at the first good pivot, so it contains 2 partitions on average. By linearity of expectation:

$$\mathbb{E}[T(n)] = \sum_{j\geq 0} 2c\left(\frac{3}{4}\right)^{j} n = 8cn = O(n)$$

The CLRS recurrence E[T(n)] ≤ (2/n) Σ E[T(k)] over k from n/2 to n−1, plus O(n), gives the same result with a sharper constant. For the median, the exact expected number of comparisons with a two-way partition is about 2(1 + ln 2)n ≈ 3.39n.
Median of Medians involves no randomness, so its average over inputs is governed by the same recurrence as its worst case which is Θ(n). What changes on typical input is only the constant. The worst-case analysis assumes the search always continues into a side of 7n/10 elements, but on random data the median of medians sits close to the true median, so the surviving side is nearer n/2. The recurrence then becomes T(n) ≈ T(n/5) + T(n/2) + cn, which sums to about cn / (1 − 0.7) ≈ 3.3cn rather than the worst-case 10cn.

- **Worst Case:** For Randomized QuickSelect, the worst case is Θ(n^2), but almost never. The worst case occurs when every random pivot is the smallest or largest element of the current range, so each partition removes just one element. The work is then

$$n + (n-1) + (n-2) + \cdots . + 1 = \frac{n(n-1)}{2}$$

A deterministic pivot hits this on every sorted array. With a random pivot, though, no input can force it: on a range of m elements, the chance of picking an extreme is only 2/m, so the chance of doing it at every step is at most the product of 2/m over the sizes visited. For a median search on n = 100 that is already below 10^−78, and the probability of running more than a constant multiple of the expected time falls off exponentially.
The worst case for Median of Medians would be a pivot that removes very few elements, but the pivot rule makes that impossible. Let x be the median of the ⌈n/5⌉ group medians. At least half of those groups have a median ≥ x, and each of them (except possibly x's own group and the last, partial group) contributes 3 elements ≥ x: its median and the two above it. So the number of elements ≥ x is at least

$$3\left(\left\lfloor\frac{1}{2}\left\lceil\frac{n}{5}\right\rceil - 2\right\rfloor\right) \geq \frac{3n}{10} - 6$$

By symmetry at least 3n/10 − 6 elements are ≤ x, so whichever side the search continues into has at most 7n/10 + 6 elements. The three-way partition preserves this: the strictly-smaller and strictly-larger blocks are each bounded by 7n/10 + 6, and the equal block ends the search. Each call does O(n) work, one recursive call on the ⌈n/5⌉ medians and one on the surviving side:

$$T(n) \leq T\left(\left\lceil\frac{n}{5}\right\rceil\right) + T\left(\frac{7n}{10} + 6\right) + an$$

Guessing T(n) ≤ cn and substituting gives

$$T(n) \leq c\left(\frac{n}{5} + 1\right) + c\left(\frac{7n}{10} + 6\right) + an = cn - \left(\frac{cn}{10}\right) - 7c - an$$

which is ≤ cn whenever cn/10 − 7c ≥ an. For n ≥ 140 that holds with c ≥ 20a, and smaller n is a constant, so T(n) = Θ(n) even in the worst case. The key fact is that the two subproblem fractions sum to less than 1: 1/5 + 7/10 = 9/10, so the total work is the

geometric series $an + (9/10)an + (9/10)^2an + \ldots \leq 10an$. With groups of 3 the fractions would be 1/3 + 2/3 = 1, every level would cost the same, and the result would be O(n log n); groups of 7 or more also give O(n).

iii. Edge Cases and Design choices:

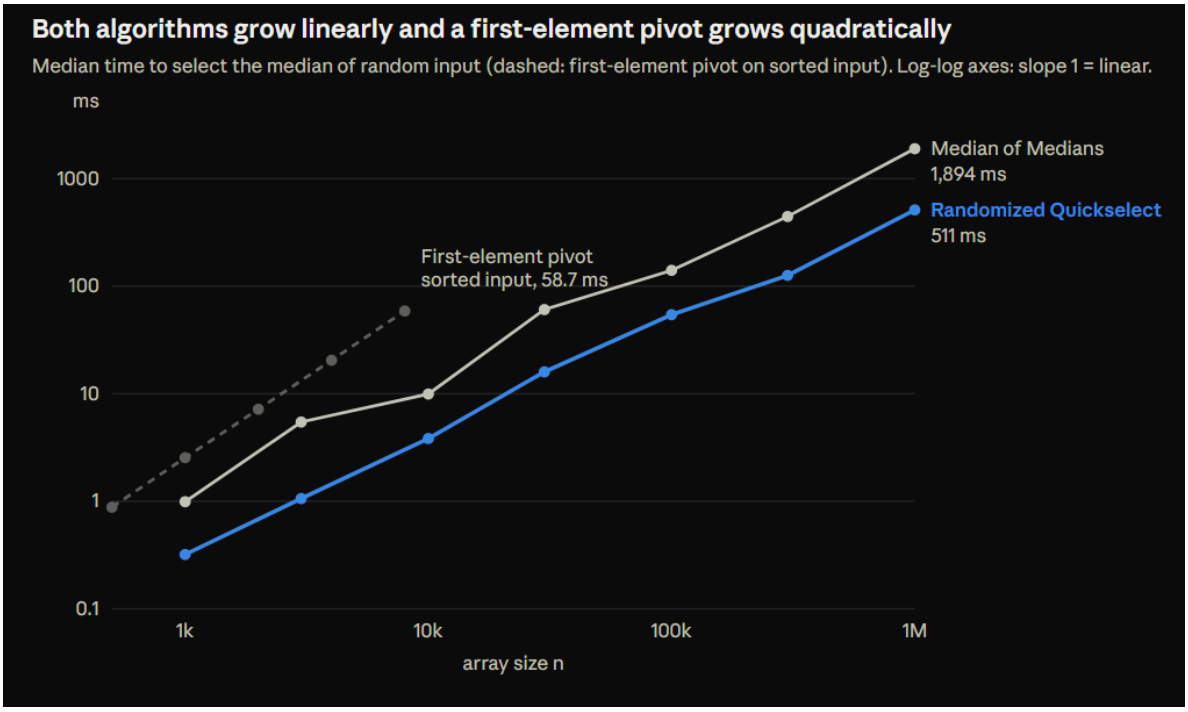
The most important edge case is duplicate keys, and the three-way partition from takes care of it, where every round either shrinks the range or ends the search in the equal block. This is why the tests deliberately include all-equal arrays and arrays with only four distinct values. Beyond that, a few smaller choices keep the functions safe and quick. Since asking for the 0th or (n+1)th smallest element is meaningless, an empty array or a k outside [1, n] raises a ValueError rather than returning something misleading. And because both functions partition a private copy of the input, callers can select from a list without finding it reordered afterwards. Partitioning also stops paying off once a range gets small, so anything under 32 elements is simply insertion-sorted. Recursion is kept to a minimum as well: narrowing down to one side happens in a loop, which leaves the median-of-medians call as the only recursive step, and since that call works on a fifth of the range each time, the stack never grows beyond O(log n).

iv. Space Complexity and Additional Overheads:

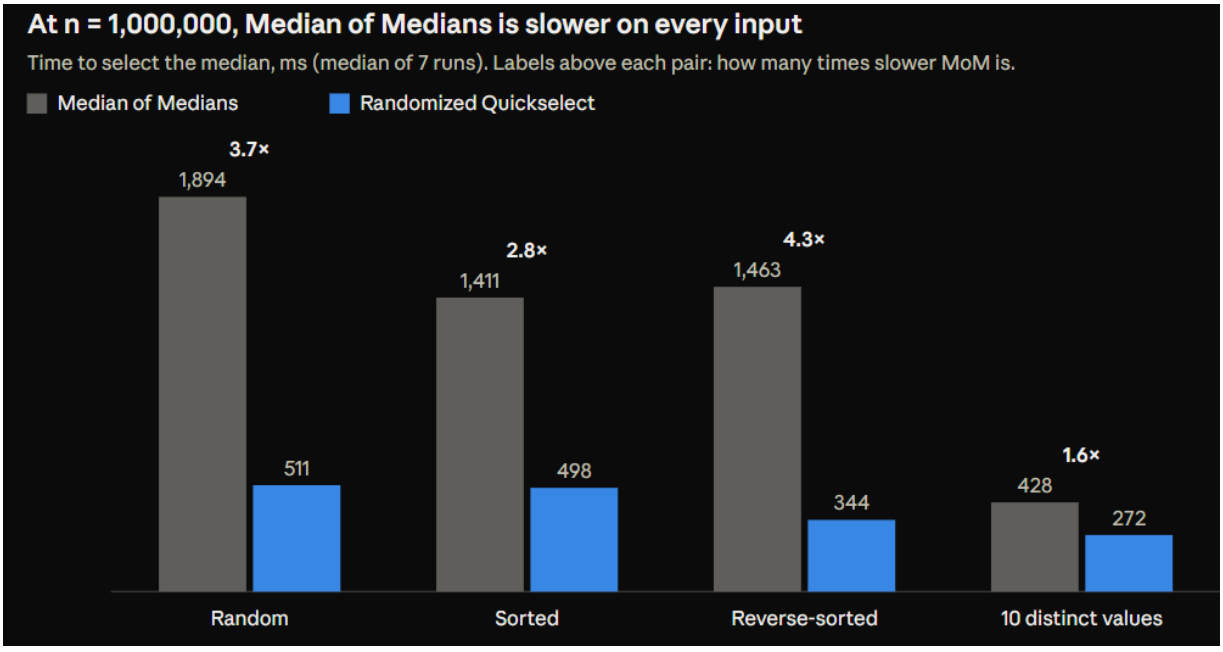
Both algorithms work in place on a single O(n) copy of the input, which exists only so that the caller's list is never reordered; dropping it would save n slots at the cost of modifying the input. Beyond that copy, randomized Quickselect needs just O(1) extra space, since its partition swaps elements in place and its narrowing step is a plain loop with no recursion. Median of Medians also partitions in place and moves each group's median to the front of the range instead of copying it into a new list, which avoids the O(n) extra space per level of a naive textbook version. It's one real cost is recursion: finding the median of the $\lceil n/5 \rceil$ medians is a recursive call on a fifth of the range, so the stack grows to about $\log_5 n$ frames, under 9 levels for n = 1,000,000, giving O(log n) space. The other overheads are constant factors rather than asymptotic ones. Median of Medians insertion-sorts every group of five, recurses on the medians and performs extra swaps before each useful partition, and its nested function calls are relatively expensive in Python. Randomized Quickselect pays only one random-number call per partition; its real downside is variance in running time.

v. Empirical Analysis:

Both algorithms ran in linear time on every input tested, and randomized Quickselect was consistently the faster of the two. Using benchmark.py under CPython 3.13 on a shared 2-core Xeon, where timings vary by about ±15%, each algorithm selected the median of arrays from 1,000 to 1,000,000 elements drawn from four distributions: random, sorted, reverse-sorted, and integers 0–9. Each setting was repeated on 7 fresh inputs and the median time reported, while a first element pivot Quickselect on sorted input served as a quadratic contrast.



On log-log axes both curves rise with a slope close to 1: a 1,000× increase in n raised the time by about 1,600× for randomized Quickselect and 1,900× for Median of Medians, with the excess coming from cache effects rather than the algorithms themselves. The first-element pivot, by contrast, nearly tripled its time with each doubling of n and already needed 59 ms at just 8,000 elements.



Input order barely mattered: sorted and reverse-sorted arrays ran about as fast as random ones, while heavy duplicates made both algorithms 2–4× faster, because the target usually lands in the three-way partition's equal block within a round or two. Median of Medians was 2.8–4.3× slower on distinct keys but only 1.6× slower with duplicates, since early exits spare it most of its costly pivot searches. Finally, 200 runs of randomized Quickselect on one 100,000-element array averaged 38.5 ms with a standard deviation of 29% of the mean, yet even its slowest run, at 72 ms, beat Median of Medians's 127 ms average.

vi. Relating results with Theory:

The measurements agree with the theory throughout. Both algorithms grew linearly, as Θ(n) predicts, and the slight extra growth at large n reflects memory caching, which the RAM model behind asymptotic analysis ignores. Randomized Quickselect never showed its theoretical $\Theta(n^2)$ worst case, consistent with that case being exponentially unlikely, whereas the first-element pivot demonstrated exactly what quadratic growth looks like when sorted input forces the worst pivot every time. Input order made little difference because Median of Medians's pivot rule depends only on values and randomized Quickselect's randomness is internal, so its expected bound holds for every input. Duplicates sped both algorithms up through the three-way partition's early exit, where a two-way partition would have slowed down instead. The constant factors line up as well, we estimated Median of Medians's comparison bound at roughly four times randomized Quickselect's, and the measured gap was 2.8–4.3× on distinct keys. Median of Medians buys its guarantee with group sorting, recursive pivot selection and extra swaps, while randomized Quickselect avoids that cost in exchange for modest run-to-run variance that never made it slower than Median of Medians in practice. For general use, then, randomized Quickselect is the better choice, and Median of Medians is best kept for settings that need a hard guarantee, such as real-time systems or adversarial inputs. Real libraries combine the two: C++'s std::nth_element uses introselect, which runs Quickselect and falls back to a guaranteed-linear method only when progress stalls.

B) Elementary Data Structures:

i. Implementation:

- Arrays and Matrices: The DynamicArray class keeps a block of memory, a capacity, and a count of how many slots are in use. Reading or writing position i is a single step, because the computer can jump straight to that slot. Adding to the end is also quick while there is spare room. When the block is full, the array creates a new block twice as large and copies everything over. That copy is slow, but it happens so rarely that adding to the end is still fast on average. Inserting or deleting in the middle is different: every element after that position has to shift by one slot to make or close the gap.
  Deletion works the same way in reverse: it shifts later elements left to close the gap. If the array falls to a quarter full, it shrinks to half its size so that it does not waste memory.
  The Matrix class stores a grid of rows and columns in one flat block, one row after another. The element in row r and column c sits at position r × cols + c, so reading or writing any cell takes a single step. Adding or removing a whole row or column changes the shape of the block, so the code builds a new block and copies the cells across. The class also includes transpose and matrix multiplication.
- Stacks and Queues: A stack is "last in, first out", like a pile of plates. ArrayStack keeps its top at the end of a DynamicArray, so push and pop only touch the last slot and never shift anything.
  A queue is "first in, first out", like a line at a shop. The obvious array version takes items from position 0, but then every remaining item has to shift forward. ArrayQueue avoids this with a circular buffer. It remembers where the front is, and both ends wrap around to the start of the block when they reach the end. Nothing ever shifts, so enqueue and dequeue are fast.

- Linked List: A linked list stores each value in its own small Node, which also holds a pointer to the next node. The list keeps a pointer to the first node (the head) and the last node (the tail). Adding at the front only creates one node and points it at the old head. Adding at the back links the old tail to a new node. Neither moves any existing data.
  To insert or delete at position i, or to delete a given value, the list must first walk from the head to find the node just before that spot. Once it has that node, changing the pointers takes only a single step. Traversal follows the next pointers from head to tail, and the class also includes search by value, access by index, and in-place reversal.
- Rooted Trees: A tree node can have any number of children, but giving every node a list of children would make each node a different size. The RootedTree class uses the "left-child, right-sibling" trick instead. Every node has exactly three pointers: one to its parent, one to its first child, and one to its next sibling. To list a node's children, you go to the first child and keep following the sibling pointers. Adding a child is a single step, because the new node simply becomes the first child. The tree supports removing a whole subtree, finding the depth of a node and the height of the tree, and two traversals. Preorder traversal uses our own ArrayStack, and level-order traversal uses our own ArrayQueue, which shows the structures working together.

ii. Time Complexity:

The table below gives the cost of each operation, where n is the number of elements stored. O(1) means the cost stays the same however big the structure gets, and O(n) means the cost grows in step with its size.

Operation	Dynamic array	Singly linked list (head + tail)	Stack (array / linked)	Queue (circular array / linked)
Access by position i	O(1)	O(i)	top only: O(1)	front only: O(1)
Search for a value	O(n)	O(n)	not supported	not supported
Insert at front	O(n)	O(1)	—	—
Insert at back	O(1) amortized	O(1)	push: O(1) amortized / O(1)	enqueue: O(1) amortized / O(1)
Insert at position i	O(n – i)	O(i) to find the spot, then O(1)	—	—
Delete at front	O(n)	O(1)	—	dequeue: O(1) / O(1)
Delete at back	O(1) amortized	O(n)	pop: O(1) amortized / O(1)	—
Delete at position i	O(n – i)	O(i)	—	—
Traverse all elements	O(n)	O(n)	—	—

The array's strength comes from its layout. All the elements sit side by side in memory, so the address of element i can be computed directly, which makes access a single step. The same layout is its weakness: to put something at the front or in the middle, every later element must move over by one slot. Adding at the back is cheap because nothing needs

to move, and the doubling rule keeps that cheap on average. Each time the array doubles, it copies all n elements, but it then has n free slots before it needs to copy again. Spread over those n additions, the copying works out to about one extra step each, which is why we call it O(1) amortized.

A linked list has no layout to exploit. To reach element i, you have to start at the head and follow i pointers, so access and search are O(n). Once you are at the right node, though, inserting or removing next to it only means changing one or two pointers, with no shifting at all. That is why adding or removing at the front is O(1). Removing the last node is the one surprise: because each node only points forward, the list has to walk the whole way from the head to find the node before the tail. A doubly linked list, with a backward pointer in every node, would fix that at the cost of more memory.

For the matrix, reading or writing a cell is O(1), because its position is computed from the row and column. Inserting or deleting a whole row or column costs O(rows × cols), since the block is rebuilt. Transposing is also O(rows × cols), and multiplying an a × b matrix by a b × c matrix costs O(a × b × c). In the rooted tree, adding a child is O(1), and finding a node's depth costs as many steps as the node is deep. Traversing the whole tree visits each node once and is O(n). Removing a subtree costs as many steps as there are siblings to search through and nodes to count.

Stacks and queues are fast because they only ever touch their ends. A stack on an array grows and shrinks at the back, which is cheap. A queue on an array needs the circular buffer to avoid shifting. A naive array queue that removes from position 0 makes every dequeue O(n), so a full run of n enqueues and n dequeues becomes $O(n^2)$. The linked versions are O(1) in every case, not just on average, because they never resize.

iii. Arrays versus linked lists for stacks and queues

Both designs give O(1) push, pop, enqueue and dequeue, so the choice between them comes down to the details of speed, memory and simplicity.

The biggest difference is how predictable each operation is. A linked stack or queue does the same small amount of work on every call: create one node, change one or two pointers. An array-based one is usually even cheaper, but every so often it fills up and has to copy everything into a bigger block. On average this costs almost nothing, yet that single copy can take a noticeable Median of Mediansent when the structure is large. For most programs this does not matter. For something like an audio player or a game loop, where every operation must finish within a strict time limit, the steady behaviour of a linked structure can be worth having.

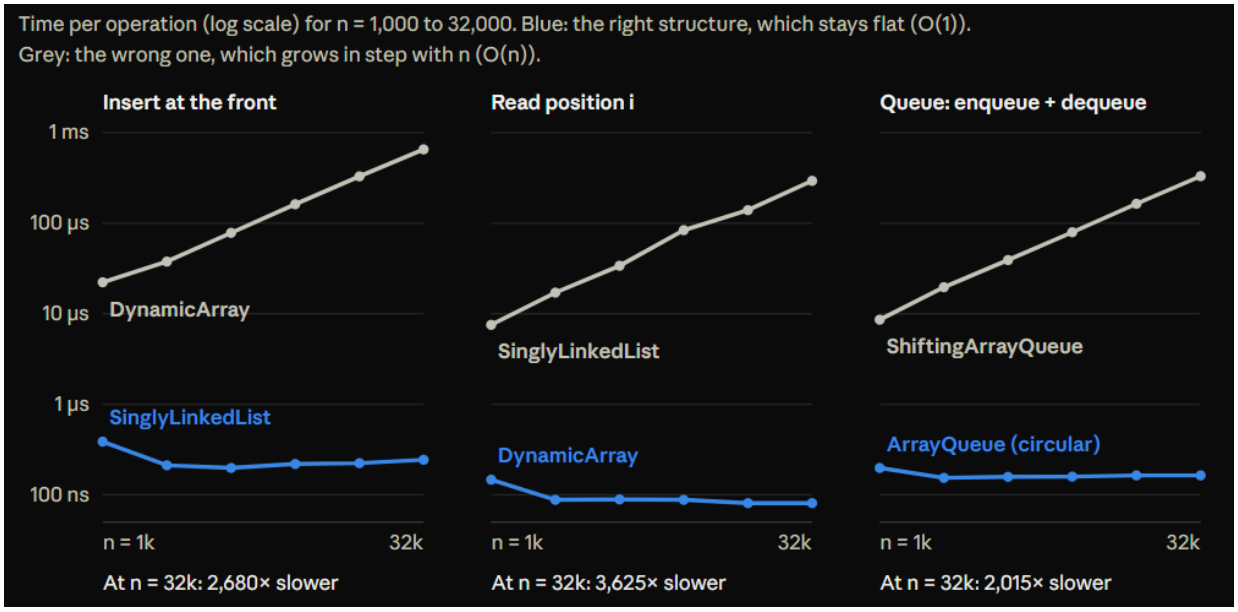
Memory points the other way. An array slot holds just the value, or a reference to it, while every linked node also carries a pointer to the next node and the extra bookkeeping of being a separate object. In our measurements, storing 100,000 integers took about 42 bytes per element in the dynamic array and about 80 bytes per element in the linked list, nearly twice as much. The array does waste some room, since after doubling it can be up to half empty, but even so it usually comes out ahead. Arrays also have an advantage on real hardware that Python mostly hides: elements that sit next to each other in memory are read much faster by the processor's cache than nodes scattered across memory. In languages like C, C++ or Java, this often makes array-based stacks and queues clearly faster.

Ease of implementation is roughly a tie, but each design has its own trap. An array stack is the simplest of all, since it is just "add and remove at the end". An array queue is the trickiest, because the obvious version shifts every element on each dequeue, and the fix needs the wrap-around arithmetic of a circular buffer and careful copying when it grows. Linked versions are short and easy to read, but they invite pointer mistakes, such as forgetting to reset the tail when the last item is removed.

In practice, array-based stacks and circular-buffer queues are the usual default, because they use less memory and work well with the cache. Linked versions are the better fit when the worst-case time of a single operation matters, when the size is completely unpredictable, or when items are already stored as nodes elsewhere in the program and can be linked in without copying.

iv. Efficiency in Specific Scenarios:

The measurements confirm the theory: picking the structure that matches the job keeps every operation at a steady cost, while picking the wrong one makes each operation slower as the data grows. The timings come from benchmark_ds.py, which runs each scenario at sizes from 1,000 to 32,000 elements, repeats it 5 times and reports the median time per operation. Exact numbers depend on the machine, but the patterns do not.



In the three cases shown, the grey line roughly doubles every time n doubles, which is what O(n) per operation looks like, while the blue line stays flat. Inserting at the front of a linked list took about 0.2 microseconds no matter how long the list was, but inserting at the front of the array slowed to 0.65 milliseconds at 32,000 elements because everything had to shift. Reading a random position flipped the result: the array answered in about 80 nanoseconds, while the linked list had to walk an average of 16,000 nodes. The queue case shows why the circular buffer matters, since the naive shifting queue was about 2,000 times slower at 32,000 items.

When both structures are O(1), the differences are small. Push and pop took about 140 nanoseconds per operation on the linked stack and about 280 on the array stack, and the circular array queue and the linked queue were both close to 160 nanoseconds. The linked stack wins here only because of Python itself: each ArrayStack call passes through a second method on DynamicArray, and the occasional resize copies elements one by one in Python code. In a compiled language the array version would usually be the faster of the

two. Memory was the clearest win for arrays: storing 100,000 integers took about 42 bytes per element in the dynamic array and 80 in the linked list.

v. Practical Applications

- Arrays are everywhere because most programs mainly read data by position. Images are stored as arrays of pixels, spreadsheets and game boards as grids, and audio as long arrays of samples. Matrices in particular are the backbone of graphics, where every rotation or zoom of a 3-D scene is a matrix multiplication, and of machine learning, where a neural network is mostly large matrices being multiplied together. Lookup tables, such as the days in each month or a price list, are arrays too. The dynamic array is the general-purpose list in most languages: Python's list, Java's ArrayList and C++'s vector all use the same doubling idea built in this report.
- Stacks appear whenever the most recent item must be handled first. The "undo" button in an editor keeps past actions on a stack and pops the latest one. A web browser's back button works the same way. Programming languages use a call stack to remember where each function should return to, and compilers use stacks to check that brackets are balanced and to evaluate expressions such as 3 + 4 × 2. Depth-first search through a maze or a file system also uses a stack, which is exactly how the tree's preorder traversal works.
- Queues appear whenever things must be handled in the order they arrived. A printer works through its print jobs in order, web servers queue incoming requests, and an operating system queues programs waiting for their turn on the processor. Streaming video and keyboard input both use circular buffers, the same design as ArrayQueue, so that data flows in at one end and out at the other without ever being shifted. Breadth-first search, which finds the shortest route in a map or the fewest connections between two people in a social network, uses a queue, as the tree's level-order traversal does here.
- Linked lists suit data that changes shape often. Music players use them for playlists where songs are added and removed in the middle, and caches that throw out the least recently used item keep their entries in a linked list so that any entry can be moved to the front in one step. Operating systems track free blocks of memory with linked lists, and hash tables often keep the items that land in the same bucket in a short linked list. Rooted trees, finally, model anything with a hierarchy: folders inside folders, the structure of a web page, a company organisation chart, or the family tree of species.

vi. Choosing Structures:

- The choice usually comes down to three questions. The first is how the data will be used. If the program mostly reads by position or loops through everything in order, an array is the clear choice. If it constantly adds and removes items in the middle and already knows where, a linked list fits better. If it only ever works with the newest or the oldest item, a stack or queue makes the intent clear and keeps every operation fast.
- The second question is memory. Arrays store values side by side with no extra pointers, so they use less memory, about half as much as the linked list in our test. They also make better use of the processor's cache. On a phone, an embedded device or when storing millions of items, this favours arrays. The exception is when the final size is

completely unknown and memory is tight, since a dynamic array can sit up to half empty just after it doubles, while a linked list uses exactly as many nodes as it holds.

- The third question is ease of implementation and how predictable the speed must be. A dynamic array or array stack is the easiest to write correctly, and in most languages it is already built in. Linked structures take more care with pointers, but they never pause to resize, which matters in real-time software such as audio, games or control systems. In practice the sensible default is to start with the built-in dynamic array, use a stack or queue when the problem naturally works with the newest or oldest item, and reach for a linked list only when frequent changes in the middle or a strict limit on each operation's time make it worth the extra memory.

Repository : https://github.com/shreeya-up/MSCS532_Assignment6.git